Ripasso di firmware per la verifica

Lezione 4 — 2MAK, SST-INF-01c — Ingressi e uscite, operatori, costrutti, funzioni

12/11/2026

Nota

Questa dispensa raccoglie il **ripasso** di firmware fatto in aula prima della verifica. È un **ripasso**, non una spiegazione nuova: i dettagli sono nelle dispense **A01**, **A01bis**, **A02** e **A03**. Per la verifica si studiano **quelle quattro più questa**. Il notebook di NotebookLM della classe contiene ora anche la A03; ognuno può crearsi anche un **notebook personale**, con le dispense di tutte le materie.

1 Ingressi e uscite: `pinMode`, `digitalWrite`, `digitalRead`

1.1 Decidere la direzione: `pinMode`

Definizione

Per dire ad Arduino se un piedino è un **ingresso** o un'**uscita** si usa **`pinMode`**: tra parentesi il **numero del piedino**, una virgola e la **direzione**, `INPUT` o `OUTPUT`, scritta **tutta in maiuscolo**. Si chiude la tonda e si mette il **punto e virgola**.

```
pinMode(6, OUTPUT);
```

Sull'**Arduino Uno** i piedini digitali che usiamo vanno dal **2 al 13** (lo 0 e l'1 servono alla comunicazione seriale). L'**Arduino Mega** ha molti più piedini digitali (54, numerati da 0 a 53): il comando è lo stesso, cambia solo il numero che si può scrivere.

1.2 Scrivere un'uscita: `digitalWrite`

Definizione

Se il piedino è un'uscita, **`digitalWrite`** lo accende o lo spegne: tra parentesi il **numero del piedino**, una virgola e **`HIGH`** (acceso) o **`LOW`** (spento), sempre in maiuscolo.

```
digitalWrite(6, HIGH);
```

1.3 Leggere un ingresso: `digitalRead`

Definizione

Se il piedino è un ingresso, **`digitalRead`** lo **legge**: tra parentesi va **solo il numero del piedino**. Di solito il valore letto si **salva in una variabile**.

```
pinMode(7, INPUT);
int x = digitalRead(7);
```

Attenzione

Dentro `digitalRead` **non si scrive `INPUT`**: la direzione l'abbiamo già decisa con `pinMode`. C'è **un solo parametro**, il numero del piedino; se il piedino è alto o basso sarà `digitalRead` a dircelo. Con questi tre comandi Arduino può **«parlare»** (uscite) e **«sentire»** (ingressi).

2 L'uguale: assegnazione e confronto

Definizione

In C il simbolo **=** è l'**assegnazione**: **copia il valore a destra** nella **variabile a sinistra**. In `x = digitalRead(7);` il valore letto finisce in `x`; in `a = 5 + 3;` il risultato 8 finisce in `a`.

Attenzione

In C l'uguale non si può girare. In matematica $5 + 3 = 8$ e $8 = 5 + 3$ sono la stessa cosa. In C `5 + 3 = a;` è un **errore**: a sinistra dell'uguale ci deve essere una **variabile**, e il nome di una variabile **non può contenere simboli** come + (è ammesso solo il trattino basso _) e **non può iniziare** con un **numero**. «Copiare 8 dentro 5+3» non ha senso.

Definizione

Il **doppio uguale** **==** è un'altra cosa: è un **confronto**, una **domanda** («sono uguali?»). Risponde vero o falso e **non cambia** nessuna variabile. Per questo `a == b` e `b == a` sono la **stessa domanda**.

3 Gli operatori

Gli **operatori** sono simboli che applicano un'elaborazione ai dati.

3.1 Operatori aritmetici e modulo

Ci sono le operazioni di base: `+`, `-`, $*$ (per), `/` (diviso). In più c'è il simbolo **`%`**, che nel nostro mondo non si chiama «percento» ma **modulo**.

Definizione

Il **modulo** `%` dà il **resto** di una **divisione tra interi**.

Esempio

- `5 / 2` **tra interi fa 2**, non 2,5: tra due interi la divisione non ha la virgola. Infatti $2 \cdot 2 = 4$, e per arrivare a 5 **manca 1**.
- `5 % 2` **fa 1**: è proprio quel resto.
- `100 % 49` **fa 2**: il 49 nel 100 ci sta **due volte** $(2 \cdot 49 = 98)$ e **avanza** 2.

Attenzione

L'errore tipico. Alla domanda «quanto fa 5 diviso 2?» molti rispondono 2,5 (la divisione con la virgola) oppure 1 (che è il **modulo**, cioè la risposta a un'altra domanda). Tra **interi**, `5 / 2` fa **2** e `5 % 2` fa **1**.

Nota

Pari o dispari. Il modulo 2 dice subito se un numero è **pari** o **dispari**: se `n % 2` vale **0** il numero è pari, se vale **1** è dispari.

3.2 Operatori di confronto

Operatore	Significato
>	maggiore
<	minore
>=	maggiore o uguale
<=	minore o uguale
==	uguale (confronto)
!=	diverso

3.3 Operatori logici e operatori bit a bit (*bitwise*)

	logico	bit a bit
AND	&&	&
OR	||	|
NOT	!	~

Definizione

Gli **operatori logici** (&&, ||, !) guardano ogni numero **tutto intero** come **vero o falso**: qualunque numero **diverso da zero** è **vero** (anche i negativi), e l'**unico** numero falso è lo **zero**. Il risultato è 1 (vero) o 0 (falso).
Gli **operatori bit a bit** (&, |, ~) invece confrontano **ogni singolo bit** del primo numero con il **bit corrispondente** del secondo, **in colonna**.

```
char a = 5;
char b = 10;
char c = a && b;
char d = a & b;
```

Esempio

Due risultati diversi.

- `c = a && b`: 5 è diverso da zero, quindi vero; 10 è vero; vero AND vero fa vero: **`c` vale 1**.
- `d = a & b`: la macchina scrive i numeri in binario e fa l'AND **colonna per colonna**:

```
  a = 5   0000 0101
 b = 10   0000 1010
-----------------------
  a & b   0000 0000 → 0, falso
```

In nessuna colonna ci sono due 1, quindi **`d` vale 0**. Stessi numeri, operatori diversi: 1 contro 0. Il 5 e il 10 sono stati scelti apposta perché hanno gli 1 in **posizioni tutte diverse**, così la differenza salta all'occhio.
Se invece `b` vale **14**:

```
  a = 5   0000 0101
 b = 14   0000 1110
-----------------------
  a & b   0000 0100 → 4
```

`a && b` fa sempre 1, mentre `a & b` fa **4** (che, essendo diverso da zero, come condizione vale vero). Sono due calcoli **completamente diversi**.

3.4 Il bit a bit nel programma del 595

L'AND bit a bit lo usiamo nel programma che collega il **74HC595** ad Arduino (dispense A02 e A03). Dentro il `for` che spedisce gli 8 bit c'è:

```
for (int i = 0; i < 8; i++)
{
  digitalWrite(SER, d & (1 << i));
  delay(1);
  carica();
}
```

SER è il piedino 3, il **filo dei dati**; `d` è il dato da spedire. `1 << i` è il numero 1 **spostato a sinistra** di `i` posizioni: a ogni giro l'1 **scorre** di un posto e **«pesca»** un bit diverso di `d`. Se quel bit è 1, `d & (1 << i)` è diverso da zero (vero) e la linea **SER va alta**; se è 0, il risultato è 0 e la linea va **bassa**.

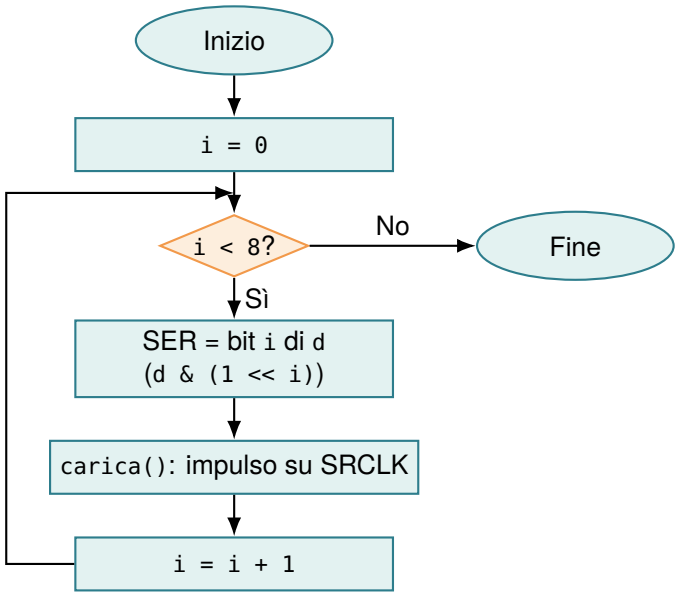


Figura 1: Il `for` che spedisce gli 8 bit di `d` nel 595.

4 I costrutti: `if`, `if ... else`, `for`, `while`

4.1 La scelta: `if` e `if ... else`

Definizione

L'**`if`** permette di fare una **scelta**: tra parentesi si scrive una **condizione** che può essere vera o falsa (per esempio `a > b`, `a == b`). Il codice tra le graffe viene eseguito **solo se** la condizione è **vera**; se è falsa, viene **saltato**.
Con **`if`** ... **`else`** ci sono **due strade**: se la condizione è vera si esegue il primo blocco, **altrimenti** (`else`) il secondo.

```
if (a > b)
{
  // eseguito se la condizione e' vera
}
else
{
  // eseguito se la condizione e' falsa
}
```

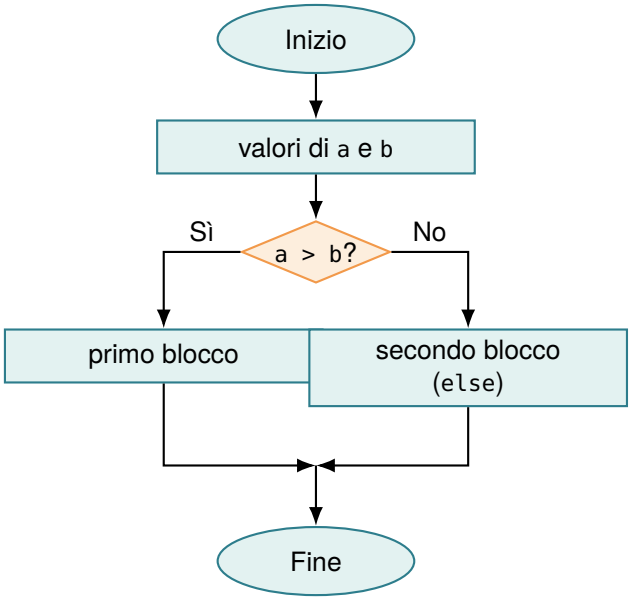


Figura 2: `if ... else`: due strade, una sola viene percorsa.

4.2 La ripetizione: `for` e `while`

Definizione

`for` e **`while`** servono a **ripetere** un pezzo di codice: invece di scriverlo 100 volte, si dice ad Arduino di ripeterlo 100 volte. Fanno **la stessa cosa**.

Il **`for`** ha tre parti tra le tonde, separate dal punto e virgola: la **condizione iniziale** (`int i = 0`), la **condizione per continuare** (`i < 10`) e il **modificatore**, cioè come `i` si avvicina al traguardo (`i++`). Tra le graffe va il codice da ripetere.

```
for (int i = 0; i < 10; i++)
{
  // codice da ripetere 10 volte
}
```

Con il **`while`** le stesse tre parti stanno in tre posti diversi: la variabile si crea **prima**, tra le tonde va solo la **condizione**, e il **modificatore** si scrive **alla fine**, prima di chiudere la graffa (se lo si dimentica, il ciclo non finisce più).

```
int i = 0;
while (i < 10)
{
  // codice da ripetere 10 volte
  i++;
}
```

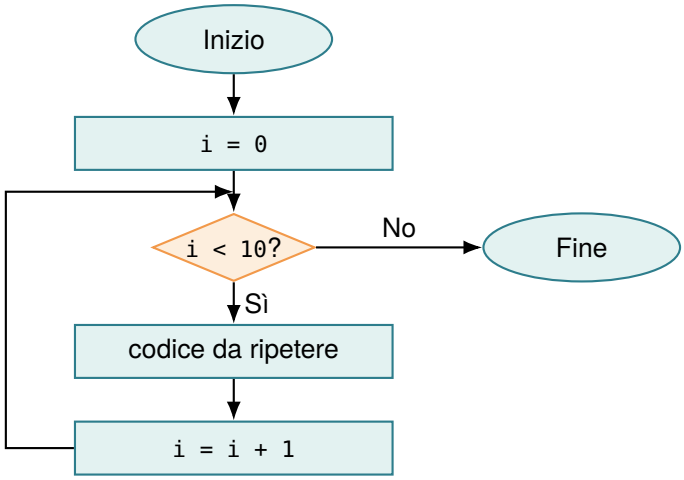


Figura 3: Lo stesso diagramma vale per il `for` e per il `while` qui sopra.

Nota

Quale scegliere? Si può usare quello che si preferisce: il `for` è un po' più **compatto**. Di solito si usa il **`for`** quando si sa **prima** quante volte ripetere (far lampeggiare un LED 10 volte) e il **`while`** quando **non si sa**, e si deve continuare **finché succede qualcosa**, per esempio finché la lettura di un **sensore** non dice di smettere.

5 Le funzioni

L'unica vera novità di quest'anno sono le **funzioni**.

Definizione

Una **funzione** è un **programma nel programma**: un gruppo di istruzioni, racchiuso tra graffe e con un **nome**, che si può **richiamare** quante volte si vuole. Le si potrebbe chiamare anche sottoprogrammi o *macro*, ma il loro nome è funzioni.

5.1 La forma più semplice

Il lampeggio di un LED sul piedino 7 sono quattro righe. Le racchiudiamo tra graffe e diamo loro un nome, `lampeggio`. Prima del nome si scrive il **tipo del dato restituito**: qui non restituiamo niente, quindi **`void`**. Tra le tonde andrebbero i **parametri**: qui nessuno.

```
void lampeggio()
{
  digitalWrite(7, HIGH);
  delay(500);
  digitalWrite(7, LOW);
  delay(500);
}
```

Nel `loop` (che, a guardarlo bene, è anch'esso una funzione) basta scrivere **`lampeggio();`**: al suo posto Arduino esegue le quattro righe. Scritto tre volte, il LED lampeggia tre volte; messo dentro un `for`, lampeggia quante volte serve.

5.2 Parametri: il passaggio per valore

Tra le tonde si possono mettere delle **variabili**, i **parametri**, che vengono riempite con i numeri che passiamo nella chiamata. Al posto del 7 fisso usiamo `x`:

```
void lampeggio(int x)
{
  digitalWrite(x, HIGH);
  delay(500);
  digitalWrite(x, LOW);
  delay(500);
}
```

Ora `lampeggio(7);` fa lampeggiare il LED sul piedino 7 e `lampeggio(11);` quello sull'11. I parametri si separano con la **virgola** e possono essere quanti si vuole (`int x, int y, int z, int w...`). Si chiama **passaggio per valore** perché il numero (7, 11) viene **copiato** nella variabile `x` **ogni volta** che si chiama la funzione.

5.3 Restituire un risultato: `return`

Definizione

Una funzione può **riconsegnare** un valore alla fine: un dato, un calcolo. Al posto di `void` si scrive il **tipo** del valore restituito (`int`, `float`, `char...`) e, prima della fine, **`return`** seguito dal valore.

L'esempio della lavagna, volutamente semplice: la funzione restituisce sempre 3.

```
int lampeggio(int x)
{
  digitalWrite(x, HIGH);
  delay(500);
  digitalWrite(x, LOW);
  delay(500);
  return 3;
}

void loop()
{
  // il 7 va in x; il 3 restituito va in w
  int w = lampeggio(7);
  lampeggio(11);
}
```

Dopo `w = lampeggio(7);`, **`w` vale 3**: è il valore riconsegnato dalla funzione e copiato nella variabile. I valori viaggiano in **due direzioni**: dalla chiamata verso la funzione (il 7 che va in `x`) e dalla funzione verso la chiamata (il 3 che va in `w`). Un esempio vero è l'**ipotenusa** della dispensa A03: la funzione riceve i due cateti e restituisce il risultato del calcolo.

Nota

Perché è comodo. Una formula complicata (anche di un'altra materia) si scrive **una volta sola** dentro una funzione; poi le si passano i valori di partenza e lei restituisce il risultato. Si **automatizza** il calcolo e si riduce la probabilità di sbagliare.

6 Cosa ripassare per la verifica

- **`pinMode`**, **`digitalWrite`**, **`digitalRead`**: parametri, maiuscole, variabile per il valore letto.
- **`=`** copia da destra a sinistra; **`==`** confronta. Nomi delle variabili: niente simboli, non iniziano con un numero.
- Operatori aritmetici e **modulo**: `5 / 2` = 2, `5 % 2` = 1, `100 % 49` = 2, pari e dispari.
- Operatori di **confronto** (>, <, >=, <=, ==, `!=`).
- Operatori **logici** contro **bit a bit**: `5 && 10` = 1, `5 & 10` = 0, `5 & 14` = 4; `d & (1 << i)` nel 595.
- **`if`**, `if ... else`, **`for`**, **`while`**: come si scrivono e quando si usano.
- **Funzioni**: `void`, parametri, **passaggio per valore**, **`return`**.
- Dispense: **A01, A01bis, A02, A03** e questa A04.